

```

71     result += formatSignAndNumber(" ", form["c"][0],
""))
72     return result
73 }
74
75 func formatSignAndNumber(signPad, number, suffix
string) string {
76     if number == "" || number == "0" || number ==
"0.0" {
77         return ""
78     }
79     var sign string
80     if signPad != "" {
81         sign = signPad + "+" + signPad
82     }
83     if number[0] == '-' {
84         sign = signPad + "-" + signPad
85         number = number[1:]
86     }
87     if suffix != "" && number == "1" {
88         return sign + suffix
89     }
90     return sign + number + suffix
91 }
92
93 func EqualFloat(x, y, limit float64) bool {
94     if limit <= 0.0 {
95         limit = math.SmallestNonzeroFloat64
96     }
97     return math.Abs(x-y) <=
98         (limit * math.Min(math.Abs(x), math.Abs(y)))
99 }
100
101 func solve(floats [3]float64) (float64, float64) {
102     a, b, c := floats[0], floats[1], floats[2]
103
104     if (b * b) - (4 * a * c) < 0 {
105         // Make x1, x2 complex numbers so
that
106         // math.IsNaN() is true ("not a float
number")
107         x1 := math.Sqrt(-1)
108         x2 := math.Sqrt(-1)
109         return x1, x2
110     } else {
111         root := math.Sqrt((b * b) - (4 * a * c))
112         x1 := (-b + root) / (2 * a)
113         x2 := (-b - root) / (2 * a)
114         return x1, x2
115     }
116 }
117
118 func processRequest(request *http.Request) ([3]
float64, string, bool) {

```

```

119     var floats [3]float64
120     count := 0
121
122     for index, key := range []string{"a", "b", "c"} {
123         if slice, found := request.Form[key]; found
&& len(slice) > 0 {
124             if slice[0] != "" {
125                 if x, err := strconv.
ParseFloat(slice[0], 64);
126                     err != nil {
127                         return floats, "" + slice[0] + ""
is invalid", false
128                     } else {
129                         floats[index] = x
130                     }
131                 } else {
132                     // The program treats
blanks as 0
133                     request.Form[key][0] = "0"
134                     floats[index] = 0
135                 }
136                 count++
137             }
138         }
139         if count != 3 { // the first time the form is
empty;
140             return floats, "", false
141         }
142         if EqualFloat(floats[0], 0, -1) {
143             return floats, "The factor of x^2 cannot not be
0", false
144         }
145         return floats, "", true
146     }
147
148     func formatSolutions(x1, x2 float64) string {
149         exactlyOneSolution := false
150         if math.IsNaN(x1) && math.IsNaN(x2) {
151             return noSolution
152         }
153         if math.IsNaN(x1) {
154             exactlyOneSolution = true
155             x1 = x2
156         } else if math.IsNaN(x2) || EqualFloat(x1, x2,
-1) {
157             exactlyOneSolution = true
158         }
159         if exactlyOneSolution {
160             return fmt.Sprintf(oneSolution,
formatSolution(x1))
161         }
162         return fmt.Sprintf(twoSolutions,
formatSolution(x1), formatSolution(x2))
163     }

```